National Autonomous University of Mexico

Faculty of Engineering

Electrical Engineering Division

Course: Compilers **Group:** 5

Semester: 2026-2

Report:
Lexical Analyzer (Lexer)

Professor:

Dr. René Adrián Dávila Pérez

Student IDs:

320060829

320060829

320060829

320060829

320060829

Delivery Date:

Sunday, March 15th, 2026

Contents

1 Introducción **2**

2 Theorical Framework **2**

2.1 Lexical Analysis (Scanner / Lexer) 2

2.2 Syntactic analysis (Parsing) and CFG (CONTEXT-FREE GRAMMARS) . . 3

2.3 Parsing Strategies: Top-Down Approach and Precedence Resolution 3

3 Desarrollo **4**

4 Resultados **4**

5 Conclusiones **4**

Referencias **4**

1 Introducción

Este apartado debe abordar los siguientes puntos:

- **Planteamiento del problema.** Se hace una descripción del problema a resolver.
- **Motivación.** Se describe porqué es necesario dar solución al problema.
- **Objetivos.** Lo que se se espera obtener de darle solución al problema.

2 Theorical Framework

Firstly, the traduction of a high level programming languaje into a low level one is a process structured in multiple phases.

In this proyect we will aboard the inicial stages of the front-end of the compiler: the lexical analysis and the syntatic analysis. This phases work together in order to validate that the source code given by the user is well formed before proceding with the semantic evaluation or the code generation. In this case, the validation procees its aplied to a code that is written in C that uyses a specific set of instructions

2.1 Lexical Analysis (Scanner / Lexer)

The lexical analysis represents the first stage of the languaje processing. Its fundamental responsability is to redd the code character by character and group them in secuences with logical meaning, known as lexemes.

Starting from this lexemes, the analyzer, generatees a secuence of tokens (as identifiers, key words, constants, special characters, operators, punctuation, literals). In the proyect, the lexical analyzer identifies specific sets as:

- **ID:** Identifiers used to give name to the variables
- **VALUE_NUM:** Literals that represent any number whole and decimal
- **VALUE_CHAR:** Literals that represent a unique character enclosed between simple commas
- **TEXT_STRING:** Text strings enclosed in double quotes, mostly used as format specifiers

2.2 Syntactic analysis (Parsing) and CFG (CONTEXT-FREE GRAMMARS)

The syntatic analyzer is the second stage in this compiler. It recibes the flow of tokens generate by the lexical analizer and verifies that the secuence fullfill the structural rules of the language.

To define this rules in a rigurous way, it uses the CFG (CONTEXT-FREE GRAMMARS), it helps to stablish which rules are valid using the production rules.

In this proyect, the CFG defines the structure to prodecue any instruction (StmtList), covering numeric declarations (DeclNum), assignments (AssignSmt), and flow control statements (lfStmt, WhileStmt, ForStmtm)

An example given is the formal rule of declaring a number (and optionally initialize it) it defines matematically in the next way:

$$DeclNum- > TypeNumIDAssignOpNum$$

The main objective of the parser is to organize the sequence of tokens into a hierarchical structure called a parse tree (or syntax tree). This structure shows how the tokens relate to each other according to the language rules and confirms that the code follows the correct syntax

2.3 Parsing Strategies: Top-Down Approach and Precedence Resolution

There are two main ways to construct the syntax tree: Top-Down and Bottom-Up. This project implements a Top-Down approach through a Recursive Descent Parser. In this method, the tree is built starting from the initial symbol of the grammar down to the leaves (the tokens), associating a recursive function with each grammar rule to process the input from left to right.

To ensure that this approach works mathematically without entering infinite loops and respecting the operation hierarchies, the grammar was designed with specific features:

- **Elimination of left recursion (Left-Recursion):** The production rules for mathematical expressions (MathExpr) were rewritten to be right-recursive.
- **Layered Precedence:** To evaluate logical conditions (Cond) while respecting the standardized hierarchy, the rules were divided into levels. The analysis starts at the lowest priority level looking for the OR logical operator (CondOrPrime), descends to the intermediate AND level (CondAndPrime), evaluates the level of relational comparisons (CondRel), and culminates at the highest priority level processing the logical NOT and parentheses (CondUnary).

3 Desarrollo

Es la descripción de la implementación realizada en el lenguaje de programación, así como las pruebas realizadas para obtener los resultados. Es importante resaltar lo siguiente:

- No se debe mostrar código, solo describir funciones o la aplicación de los conceptos teóricos.
- Las pruebas es describir las entradas ingresadas y las salidas obtenidas.

4 Resultados

Mediante capturas de pantalla y una breve descripción seguida de la captura se presentan los resultados finales de su aplicación.

5 Conclusiones

Se presenta un análisis de los resultados obtenidos, donde se destaca la importancia de la aplicación de los conceptos teóricos para resolver el problema. Es importante resaltar lo siguiente:

- No es describir si les gustó la actividad o no.
- No es decir que se obtuvo de la práctica.
- No es describir lo que fue difícil.

Referencias

[1] D. Harris and S. Harris, *Digital Design and Computer Architecture*, Arm Edition. Morgan Kaufmann Pub, Apr. 22, 2015.

[2] A. Elahi, *Computer systems, Digital Design, Fundamentals of Computer Architecture and ARM Assembly Language*, 2nd ed. Springer, Mar. 17, 2022. [Online]. Available: https://doi.org/10.1007/978-3-030-93449-1

[3] S. Smith, *Raspberry Pi Assembly Language Programming*. Apress, Oct. 2019. [Online]. Available: https://doi.org/10.1007/978-1-4842-5287-1

[4] R. Anaya. "Manual de recursos y aplicaciones plataforma rasberry pi. "[Online]. Available: https://odin.fi-b.unam.mx/micros/docs/Tutoriales%20Raspberry.pdf